

UNIVERSIDAD
SERGIO ARBOLEDA

Diseño e implementación de una gramática LL(1)

Un lenguaje con operaciones aritméticas, funciones trigonométricas y asignación de variables

Lenguajes de programación y traducción

David Andrés Castellanos Angulo

Universidad Sergio Arboleda
Escuela de ciencias exactas e ingeniería
Bogotá D.C., 27 de septiembre de 2026

1. Objetivo del trabajo

El trabajo pide diseñar e implementar una gramática LL(1) para un lenguaje de programación que soporte las operaciones `+`, `-`, `*`, `/`, `%` y el valor absoluto, las funciones `sin`, `cos` y `tan`, y la asignación de variables. Hay que garantizar las tres fases de análisis (léxico, sintáctico y semántico), mostrar los conjuntos PRIMEROS, SIGUIENTES y PREDICCIÓN, y hacer pruebas de implementación.

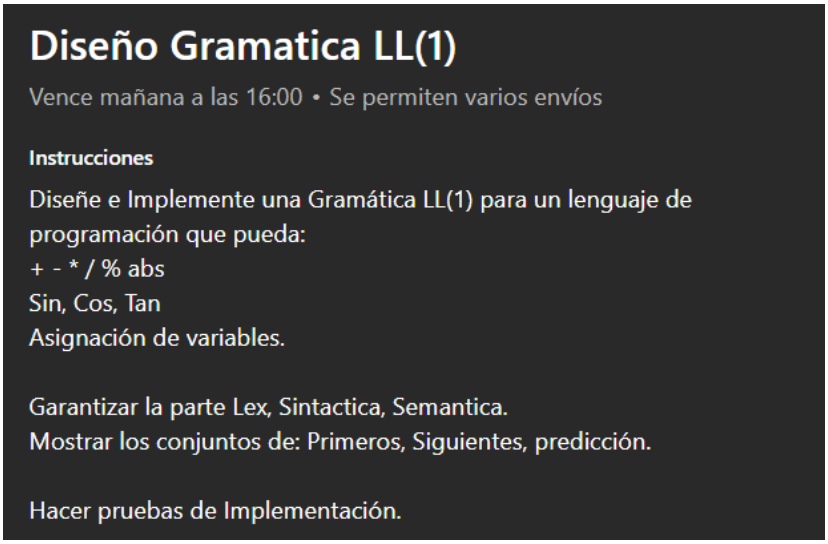


Figura 1: Enunciado del trabajo.

Lo resolví en Python, sin librerías externas, reutilizando el programa de conjuntos del taller anterior para verificar que la gramática que diseñé sí es LL(1).

2. Diseño de la gramática

2.1. Tokens del lenguaje

Antes de escribir las reglas definí qué va a entregar el analizador léxico. Los terminales de la gramática son clases de token, no lexemas sueltos: `ID` para los nombres de variable y `NUM` para los números.

Cuadro 1: Tokens del lenguaje.

Clase	Lexema	Descripción
`ID`	`base`, `x1`	nombre de variable
`NUM`	`12`, `0.5`	número entero o real
`asig`	`=`	asignación
`fin`	`;`	fin de sentencia
`mas`, `menos`	`+`, `-`	suma y resta, y el menos unario
`por`, `entre`	`*`, `/`	multiplicación y división
`modulo`	`%`	residuo entre enteros
`abre`, `cierra`	`(`, `)`	agrupación
`barra`	`|`	valor absoluto en forma `|x|`
`abs`, `sin`, `cos`, `tan`		palabras reservadas

Los comentarios empiezan con `#` y llegan hasta el fin de línea; los espacios y saltos de línea se ignoran.

2.2. Las reglas

Listing 1: `grammars/lenguaje.txt`: la gramática del lenguaje.

```
# Gramática LL(1) del lenguaje
# Los no terminales son los símbolos de la izquierda; el resto son
   terminales.
# Los terminales en MAYÚSCULA son clases de token que entrega el
   analizador léxico.

prog     -> sents
sents    -> sent sents | ε
sent     -> ID asig expr fin

expr     -> term expr_r
expr_r   -> mas term expr_r | menos term expr_r | ε

term     -> fact term_r
term_r   -> por fact term_r | entre fact term_r | modulo fact term_r |
    ε

fact     -> menos fact | unidad

unidad   -> NUM
unidad   -> ID
unidad   -> abre expr cierra
unidad   -> barra expr barra
unidad   -> func abre expr cierra

func     -> abs | sin | cos | tan
```

Tomé tres decisiones para que la gramática quedara LL(1):

- **Precedencia con niveles.** *expr* agrupa sumas y restas, *term* agrupa producto, división y módulo, y *fact* es lo que va más abajo en el árbol. Como el nivel más profundo se resuelve primero, `2 + 3 * 4` da 14 y no 20.
- **Sin recursión izquierda.** En vez de *expr* → *expr* `+` *term*, que haría que el procedimiento se llamara a sí mismo sin consumir nada, la repetición se lleva a la derecha con *expr_r* y *term_r*, que terminan en ε. La asociatividad por la izquierda se conserva al armar el árbol en la implementación.
- **Sin prefijos comunes.** Las cinco alternativas de *unidad* empiezan cada una con un token distinto (`NUM`, `ID`, `(`, `|` y una función), y las cuatro funciones

se agruparon en *func* para que *unidad* tenga una sola regla que empiece con función. Así no hay que factorizar nada.

El menos unario está en *fact* → `menos` *fact*, que no choca con la resta binaria porque esa decisión se toma en *expr_r*, en otra posición del recorrido.

3. Conjuntos y verificación LL(1)

Los conjuntos los calcula `conjuntos.py`, el mismo programa del taller anterior, con algoritmos de punto fijo.

Cuadro 2: PRIMEROS y SIGUIENTES de la gramática.

No terminal	PRIMEROS	SIGUIENTES
prog	{ID, ε}	{$}
sents	{ID, ε}	{$}
sent	{ID}	{ID, $}
expr	{ID, menos, NUM, abre, barra, abs, sin, cos, tan}	{fin, cierra, barra}
expr_r	{mas, menos, ε}	{fin, cierra, barra}
term	{ID, menos, NUM, abre, barra, abs, sin, cos, tan}	{fin, mas, menos, cierra, barra}
term_r	{por, entre, modulo, ε}	{fin, mas, menos, cierra, barra}
fact	{ID, menos, NUM, abre, barra, abs, sin, cos, tan}	{fin, mas, menos, por, entre, modulo, cierra, barra}
unidad	{ID, NUM, abre, barra, abs, sin, cos, tan}	{fin, mas, menos, por, entre, modulo, cierra, barra}
func	{abs, sin, cos, tan}	{abre}

Cuadro 3: Conjuntos de PREDICCIÓN de las 24 reglas.

#	Regla	PREDICCIÓN
1	*prog* → *sents*	{ID, $}
2	*sents* → *sent sents*	{ID}
3	*sents* → ε	{$}
4	*sent* → ID asig *expr* fin	{ID}
5	*expr* → *term expr_r*	{ID, menos, NUM, abre, barra, abs, sin, cos, tan}
6	*expr_r* → mas *term expr_r*	{mas}
7	*expr_r* → menos *term expr_r*	{menos}
8	*expr_r* → ε	{fin, cierra, barra}
9	*term* → *fact term_r*	{ID, menos, NUM, abre, barra, abs, sin, cos, tan}
10	*term_r* → por *fact term_r*	{por}
11	*term_r* → entre *fact term_r*	{entre}
12	*term_r* → modulo *fact term_r*	{modulo}
13	*term_r* → ε	{fin, mas, menos, cierra, barra}
14	*fact* → menos *fact*	{menos}
15	*fact* → *unidad*	{ID, NUM, abre, barra, abs, sin, cos, tan}
16	*unidad* → NUM	{NUM}
17	*unidad* → ID	{ID}
18	*unidad* → abre *expr* cierra	{abre}
19	*unidad* → barra *expr* barra	{barra}
20	*unidad* → *func* abre *expr* cierra	{abs, sin, cos, tan}
21	*func* → abs	{abs}
22	*func* → sin	{sin}
23	*func* → cos	{cos}
24	*func* → tan	{tan}

El criterio LL(1) es que las reglas de un mismo no terminal tengan conjuntos de predicción disjuntos. El programa lo revisa par por par y confirma que se cumple:

```
¿ES LL(1)? SÍ: las reglas de cada no terminal tienen predicciones
   disjuntas.
```

Se ve en la tabla anterior: las tres reglas de *expr_r* predicen `mas`, `menos` y el resto (`fin`, `cierra`, `barra`), sin cruzarse; y las cinco de *unidad* predicen cada una un token distinto.

4. Implementación

El trabajo quedó dividido en un módulo por fase, igual que el esquema del compilador visto en clase.

4.1. Análisis léxico

`lexico.py` recorre el texto carácter por carácter y arma la lista de tokens. Reconoce números enteros y reales, identificadores, palabras reservadas, los símbolos del lenguaje y los comentarios. Cada token guarda su línea y columna, que es lo que permite reportar los errores con la posición exacta:

```
Error léxico: línea 2, columna 7: símbolo no reconocido '@'.
```

4.2. Análisis sintáctico

`sintactico.py` es un analizador descendente recursivo: hay una función por cada no terminal de la gramática, y cada una decide qué regla aplicar mirando un solo token de anticipación, con los conjuntos de predicción de la tabla anterior. No hay retroceso.

Las funciones de *expr_r* y *term_r* reciben el operando izquierdo ya construido y van armando el árbol hacia la izquierda; por eso `10 - 3 - 2` da 5 y no 9, aunque la gramática repita por la derecha.

El resultado es el árbol sintáctico, representado con tuplas como `('bin', '*', izq, der, linea)`. Los errores también salen con posición:

```
Error sintáctico: línea 3, columna 1: se esperaba ';' y se encontró 'y
    '.
```

4.3. Análisis semántico

`semantico.py` recorre el árbol y revisa lo que la gramática no puede revisar sola. Mantiene una tabla de símbolos y aplica estas reglas:

- una variable no se puede usar si no fue asignada antes;
- `+`, `-` y `*` dan entero si los dos operandos son enteros, y real en cualquier otro caso;
- `/` siempre da real;
- `%` solo se permite entre enteros;
- no se puede dividir entre la constante cero;
- `sin`, `cos` y `tan` reciben radianes y devuelven real; `abs` y `|x|` conservan el tipo.

```
Error semántico: línea 2: la variable 'y' se usa sin haber sido
    asignada.
Error semántico: línea 2: el módulo '%' solo se puede usar entre
    enteros (se recibió real y entero).
```

Después del análisis, una segunda pasada evalúa el programa y llena la tabla con el valor de cada variable, que es lo que se imprime al final.

5. Pruebas de implementación

En `test_implementacion.py` hay 31 pruebas automáticas con `unittest`, repartidas así:

Cuadro 4: Grupos de pruebas.

Grupo	Pruebas	Qué revisa
Léxico	6	tokens, comentarios, reales, símbolos inválidos
Sintáctico	9	precedencia, asociatividad, paréntesis, menos unario, errores
Semántico	6	tipos, variable sin asignar, módulo con reales, división por cero
Funciones	3	`sin`, `cos`, `tan` y funciones anidadas
Gramática	5	LL(1), sin recursión izquierda, PRIMEROS, SIGUIENTES y PREDICCIÓN
Ejemplos	2	los archivos de `ejemplos/` dan el resultado y el error esperados

Las pruebas de precedencia y asociatividad son las que muestran que la gramática quedó bien diseñada: si los niveles *expr* / *term* / *fact* estuvieran mal, `2 + 3 * 4` daría 20. Y la prueba de la gramática verifica que no hay conflictos LL(1) sobre el archivo real, no sobre una copia escrita a mano.

Listing 2: Comandos usados.

```
chmod +x run_all.sh
./run_all.sh                        # pruebas, conjuntos y ejemplos
python3 main.py ejemplos/demo.lp    # análisis completo de un programa
python3 main.py --conjuntos         # PRIMEROS, SIGUIENTES, PREDICCIÓN
    y LL(1)
```

6. Ejemplo completo

Listing 3: `ejemplos/demo.lp`.

```
# Programa de prueba del lenguaje

base = 12;
altura = 5;
area = base * altura / 2;

resto = 17 % base;
desfase = |3 - 10|;
negativo = -base + 4;

angulo = 0.7853981634;          # 45 grados en radianes
razon = sin(angulo) / cos(angulo);
mezcla = abs(-2.5) * (base + 1) - tan(0.0);
```

Listing 4: Salida de `python3 main.py ejemplos/demo.lp`.

```
[1] Análisis léxico: 72 tokens reconocidos.
[2] Análisis sintáctico: 9 sentencias, sin errores.
[3] Análisis semántico: tipos correctos y variables asignadas antes de
    usarse.

Resultados
  base     = 12   (entero)
  altura   = 5    (entero)
  area     = 30.0   (real)
  resto    = 5    (entero)
  desfase  = 7    (entero)
  negativo = -8   (entero)
  angulo   = 0.785398   (real)
  razon    = 1.0    (real)
  mezcla   = 32.5   (real)
```

Se puede comprobar a mano: `area` es $12 \times 5/2 = 30$ y queda real porque hay una división; `resto` es 17 mód $12 = 5$ y queda entero; `desfase` es $|3 - 10| = 7$; y `razon` es $\tan(45°) = 1$ calculado como seno sobre coseno.

7. Conclusiones

- La gramática quedó LL(1) por diseño, no por casualidad: los niveles de precedencia, la repetición por la derecha y el agrupar las funciones en un solo no terminal son las tres decisiones que evitan recursión izquierda y prefijos comunes.
- Los conjuntos de predicción no son solo un ejercicio de papel: son exactamente las condiciones de los `if` del analizador descendente recursivo.
- La separación en tres módulos hace que cada error quede donde corresponde. Un símbolo raro lo detecta el léxico, un `;` faltante lo detecta el sintáctico y una variable sin asignar solo se puede detectar en el semántico, porque la gramática no tiene cómo saberlo.
- Las 31 pruebas cubren las tres fases y la gramática, así que cualquier cambio que rompa la precedencia o introduzca un conflicto LL(1) se nota de inmediato.